



天数智芯
Iluvatar CoreX

天数智芯

测试工具使用指南

版本：V4.3.0 (x86) | V4.3.6 (ARM)

日期：2025.6.30

适用产品：天垓 150 | 智铠系列

声明

版权与保密声明

本文档（属未发表作品）版权归属于天数智芯全权所有，且本声明不构成对本文档已发表的承认。本文档所载全部信息均为天数智芯保密信息（包括但不限于天数智芯技术信息、经营信息、专属于接收方的未公开资料等）。未经天数智芯事先书面许可，严禁任何形式的使用、复制、传播、转录或翻译。

免责声明

天数智芯可以随时对本文档或本文档中描述的产品进行改进和/或更改。本文档包括与天数智芯产品有关的信息，作为说明典型应用的一种方式，因此，不一定提供足以进行生产设计的完整信息。对于本文档中内容的准确性或完整性，天数智芯不做任何陈述或保证。

联系方式

地址：上海闵行区陈行公路 2168 号 3 幢

电话：021-68886607

网址：www.iluvatar.com

目录

1 测试工具使用指南	5
1.1 修订记录	5
1.2 工具概览	6
1.3 开始使用测试工具之前	7
1.4 带宽测试工具	7
1.4.1 带宽测试方法	8
1.4.2 带宽测试工具功能列表	8
1.5 矩阵算力测试工具	9
1.5.1 矩阵算力测试方法	9
1.6 P2P 性能测试工具	10
1.6.1 P2P 性能测试方法	10
1.6.2 (虚拟机环境下) P2P 性能提升工具	10
1.6.2.1 开始之前	11
1.6.2.2 P2P 性能提升方法	11
1.7 通讯库性能测试工具	12
1.7.1 开始之前	12
1.7.2 参数说明	12
1.7.2.1 mpirun 重要参数说明	12
1.7.2.2 NCCL 环境变量说明	13
1.7.2.3 MCA 参数说明	13
1.7.2.4 CUDA 环境变量说明	13
1.7.2.5 ixCCL-tests 可执行程序参数说明	13
1.7.3 通讯库性能测试方法	13
1.8 GDS 基准测试工具	14
1.8.1 开始之前	14
1.8.1.1 NVMe 磁盘与驱动	14
1.8.1.2 itrfs 驱动和 libcufile.so 文件	15
1.8.2 ixGDS 工具使用方法	15
1.8.2.1 使用 GDSIO 工具	15
1.8.2.2 使用 GDSCP 工具	16
1.9 GPU 性能调试工具	17
1.9.1 使用方法	17
1.9.2 使用示例	18
1.9.2.1 示例 1: 使用 -l, --list 查看 GPU 的状态	18
1.9.2.2 示例 2: 使用 --topo 参数查看 PCIe 拓扑图	18
1.9.2.3 示例 3: 使用 --trace 检测引起通讯的阻塞或降速的因素	20
1.9.2.4 示例 4: 使用 --disable-acs 关闭 switch 的 acs	22
1.9.2.5 示例 5: 使用 --enable-acs 开启 switch 的 acs	22
1.9.2.6 示例 6: 使用 --enable-extend 使能 PCIe 扩展能力	22
1.9.2.7 示例 7: 使用 --set-mps 设置 MaxPayload Size	22
1.9.2.8 示例 8: 使用 --set-mrrs 设置 MaxReadReq Size	23
1.9.2.9 示例 9: 使用 --set-timeoutDis 设置 Completion Timeout Disable	23

1.10 (第三方类 FPGA 设备) DMA 测试工具	23
1.10.1 场景 1: 用户设备为 Master, 天数智芯加速卡为 Slaver	24
1.10.1.1 RDMA 公共 API 接口	29
1.10.2 场景 2: 天数智芯加速卡为 Master, 用户设备 (类 FPGA 设备) 为 Slaver	30

1 测试工具使用指南

1.1 修订记录

- COREX01-BI430-UG06-00: 2025/6/30
 - 更新测试工具安装方式和默认安装路径, 新增测试工具安装包 `corex-toolbox-{ARCH}-{version}.run`
 - P2P 性能提升工具, 更新**P2P 性能提升方法**
 - 新增**GPU 性能调试工具**
- COREX01-BI420-UG06-00: 2025/1/24
 - 更新**P2P 性能测试工具** 安装包名称为 `ixgpu-vm-p2p_{version}_x86_64.run`
- COREX01-BI411SP1-UG06-00: 2024/11/28
 - 删除**矩阵算力测试方法** 和**P2P 性能测试方法** 中的算力示例数据
- COREX01-BI411-UG06-00: 2024/8/26
 - 新增**开始使用测试工具之前** 检查项目以避免带宽测试结果异常; 提示切换用户后, 必须重新设置环境变量
 - 更新**(虚拟机环境下) P2P 性能提升工具** 获取方式, 提示用户可在 [天数智芯官网](#) 下载工具包
 - 更新**通讯库性能测试工具**, 新增 Open MPI v4.0.7 为使用 ixCCL-tests 工具的前提条件
 - 更新**(第三方类 FPGA 设备) DMA 测试工具**
- COREX01-BI410-UG06-01: 2024/7/22
 - **工具概览** 补充 GDS 基准测试工具和 DMA 测试工具概述
 - 更新**rdma_itr_p2p_dma_unmap_pages** API 的信息
 - 删除重复的**rdma_itr_p2p_dma_map_pages** API
- COREX01-BI410-UG06-00: 2024/6/30
 - 更新**P2P 性能测试工具** 的结果矩阵
 - 新增**GDS 基准测试工具**, 用于验证 GDS 功能是否可用
 - 新增**(第三方类 FPGA 设备) DMA 测试工具**, 用于读写用户设备的内存
- COREX01-BI340-UG06-02: 2024/5/7
 - 新增**(虚拟机环境下) P2P 性能提升工具**, 提供 P2P 性能提升工具 `ixgpu-vm-p2p-tool` 的使用方法
- COREX01-BI340-UG06-01: 2024/4/9
 - 更新**通讯库性能测试工具**, 使用 ixCCL-tests 工具前, 需要先设置环境变量 `export NCCL_USE_DIRECT=1`
- COREX01-BI340-UG06-00: 2024/3/25

文档本次发布内容与 V3.3.0-BI150 文档相比 (COREX01-BI330-UG06-01), 有以下更新:

 - 更新测试工具默认安装路径为 `/usr/local/corex-{v.r.m}/extras/test_demo/`
 - 更新 `bandwidthTest` **带宽测试工具** 的描述, 并提供更详细的**带宽测试方法**

- 更新**矩阵算力测试方法**，支持 BF16 数据类型
- 新增**通讯库性能测试工具**，用于验证 all gather、all reduce、all to all、broadcast、reduce scatter 等多卡通信 OP 的功能和性能

1.2 工具概览

天数智芯提供以下两种测试工具安装方式：

- 随天数智算软件栈同时安装，默认路径为 `/usr/local/corex-{v.r.m}/corex-toolbox-{version}/bin/`

该安装方式适用需要同时使用天数智算软件栈和测试工具的场景。

- 执行 `corex-toolbox-{ARCH}-{version}.run` 安装包单独安装，普通用户默认安装路径为 `$HOME/corex-toolbox-{version}/bin/`，root 用户默认安装路径为 `/root/corex-toolbox-{version}/bin/`

该安装方式适用仅需要使用天数智算软件栈的场景，或前期无测试工具的使用需求但后期存在使用需求的场景。

天数智芯提供以下测试工具：

工具	描述
带宽测试工具	bandwidthTest 工具提供 GPU memcpy 带宽以及 PCIe 中的 memcpy 带宽数据，可以测试 host 和 device，以及 device 间的传输带宽。
矩阵算力测试工具	gemm_perf 工具执行用户指定数据类型、循环次数和矩阵大小的 GEMM 运算。
P2P 性能测试工具	p2pBandwidthTest 工具提供基于 DMA 的 P2P 传输性能测试。
通讯库性能测试工具	ixCCL-tests 工具用于验证 all gather、all reduce、all to all、broadcast、reduce scatter 等多卡通信 OP 的功能和性能。
GDS 基准测试工具	ixGDS 工具用于验证 GDS 功能是否可用。GDS 通过 DMA 使 GPU 与磁盘之间可以直接传输数据，避免了数据通过 CPU，可以提升数据传输带宽，降低 CPU 负载。
GPU 性能调试工具	ix-gpu-tuner 工具用于检测影响 GPU 最优性能使用的因素，例如，iommu, acs, 拓扑等。
(第三方类 FPGA 设备) DMA 测试工具	ixgpu_to_third_party_dmatetest 工具用于读写用户设备的内存。

(第三方类 FPGA 设备) DMA 测试工具需要您向应用工程师单独获取。

1.3 开始使用测试工具之前

在测试之前，请检查以下项目，以避免带宽测试结果异常：

- 关闭主机 BIOS 中的 IOMMU/SMMU 设置，并且 grub iommu off。开启 IOMMU 可能会降低 PCIe 带宽性能。
- 关闭所有 Switch 的 ACS。PCIe 交换机开启 ACS 也可能会降低 PCIe 带宽性能。
- 检查 Switch 和天数智芯加速卡的 LnkSta 速率是否为 PCIe 4.0 x16，如果存在 downgraded 的情况，会降低 P2P 带宽速率。
- 启用天数智芯加速卡对应的 PCIe 设备的 ExtTag 功能。
- NUMA 结构会影响传输带宽，相同的 NUMA node 节点下的设备与内存的读写效果会更好。
- 某些 BIOS 版本会导致带宽性能偏低。

如果您切换了用户使用测试工具 (如使用 root 用户安装软件栈后，切换 user 用户使用测试工具)，必须为切换后的用户重新设置环境变量。以软件栈默认安装路径 /usr/local/corex-{v.r.m}/ 为例，您需要：

- 为 PATH 环境变量加上 /usr/local/corex-{v.r.m}/corex-toolbox-{version}/bin
- 为 LD_LIBRARY_PATH 环境变量加上 /usr/local/corex-{v.r.m}/lib

1.4 带宽测试工具

bandwidthTest 测试工具可提供 GPU memcpy 带宽以及 PCIe 中的 memcpy 带宽数据，它能测试 device 到 device 之间的传输带宽，也可测试分页内存和页锁定内存的 host 到 device 之间的传输带宽，以及分页内存和页锁定内存的 device 到 host 之间的传输带宽。

带宽测试主要涵盖三个测试场景：

- host to device，即 H2D，从系统内存搬运指定大小 (size) 的数据到天数智芯加速卡 HBM 内存，循环 100 次。记录总时间，用 $size * 100 / \text{总时间}$ ，计算带宽。
- device to host，即 D2H，从天数智芯加速卡 HBM 内存搬运指定大小的数据到系统内存，循环 100 次。计算方式相同。
- device to device，即 D2D，从天数智芯加速卡 HBM 内存地址 A 搬运指定大小的数据到天数智芯加速卡 HBM 内存地址 B。计算方式相同。

各场景可对应如下内存模式：

场景	支持分页内存模式 (pageable)	支持页锁定内存模式 (pinned)
device to host	支持	支持
host to device	支持	支持
device to device		支持

您可以选择测试模式，包括 quick、range 和 shmoo，不同模式测试的内存范围不同。详情参考下方参数描述。

针对不同内存模式，测试工具分别调用如下函数：

- 分页内存模式：cudaMemcpy
- 页锁定内存模式：cudaMemcpyAsync

1.4.1 带宽测试方法

命令格式：

```
$ bandwidthTest < option >...
```

以下命令分别测量 host to device, device to host 和 device to device 的带宽, size 范围为 10485760 字节 (start) 至 104857600 字节 (end), 以 10485760 字节为 size 增量 (increment)。

```
$ cd /usr/local/corex-{v.r.m}/corex-toolbox-{version}/bin # 以随软件栈同时安装测试工具为例
$ ./bandwidthTest --memory=pinned --mode=range \
--start=10485760 --end=104857600 --increment=10485760 --htod
$ ./bandwidthTest --memory=pinned --mode=range \
--start=10485760 --end=104857600 --increment=10485760 --dtoh
$ ./bandwidthTest --memory=pinned --mode=range \
--start=10485760 --end=104857600 --increment=10485760 --dtod
```

start 和 end 不一样, 表示 size 逐渐递增, 每次 size 是从 start 开始, 每次以 increment 递增取 size, 每个 size 都计算一次带宽。所以第一次搬运的 size 是 10485760 (start), 第二次是 20971520 (start+increment), 第三次是 31457280 (start+increment*2), 以此类推。您可以自行设定搬运的数据量 (size), 测试时长将随数据量的增加而增加。

1.4.2 带宽测试工具功能列表

bandwidthTest 工具的可选项如下：

命令参数	命令描述
--help	Display this help menu
--csv	Print results as a CSV
--device= < deviceno >	Specify the device to be used all : Compute cumulative bandwidth on all the devices 0,1,2,...,n : Specify any particular device to be used
--memory= < memmode >	Specify which memory mode to use: pageable : Use pageable memory pinned : Use non-pageable system memory

命令参数	命令描述
--mode= < mode >	Specify the mode to use: quick : Perform a quick measurement, the default size is 32M range : Measure a user-specified range of values shmoo : Perform an intense shmoo of a large range of values
--htod	Measure host to device transfers
--dtoh	Measure device to host transfers
--dtod	Measure device to device transfers
--wc	Allocate pinned memory as write-combined
--cputiming	Force CPU-based timing always
--start= < size >	(For range mode only) The starting transfer size in bytes
--end= < size >	(For range mode only) The ending transfer size in bytes
--increment= < size >	(For range mode only) Increment size in bytes

1.5 矩阵算力测试工具

天数智芯提供以下算力测试工具：

算力测试项	工具	说明
矩阵算力	gemm_perf	用于统计计算的 GPU 总用时以及 TFlops/s。使用 gemm_perf 矩阵算力测试工具，可以指定需要测试矩阵乘的 MNK 大小，转置属性，循环次数，以及矩阵元素的数据类型。

1.5.1 矩阵算力测试方法

执行测试程序，可显示对应参数的详细说明，具体介绍如下：

```
$ cd /usr/local/corex-{v.r.m}/corex-toolbox-{version}/bin # 以随软件栈同时安装测试工具为例
$ ./gemm_perf --help
```

Iluvatar GEMM performance benchmark.

Usage: ./gemm_perf [OPTIONS]

Options:

```
-h,--help Print this help message and exit
--m UINT Specify GEMM M size.
--n UINT Specify GEMM N size.
--k UINT Specify GEMM K size.
--ta BOOLEAN Specify whether transpose matrix A.
--tb BOOLEAN Specify whether transpose matrix B.
--d UINT Specify data type of input matrices. 0 for FP32, 1 for FP16, 2 for INT8, 3 for
    BF16.
--l UINT Specify number of loops calculating GEMM.
--nd Use normal distribution to generate matrices, default using uniform distribution
--limit FLOAT Use limit value to set the fixed range, by default it is 0
```

例如，执行 100 次 FP16 矩阵相乘（不指定 m, n, k, ta, tb 等参数时默认会选择较优配置）：

```
$ ./gemm_perf --d 1 --l 100
```

1.6 P2P 性能测试工具

p2pBandwidthTest 工具提供基于 DMA 的天数智芯加速卡之间 P2P 传输性能测试。

1.6.1 P2P 性能测试方法

执行以下命令：

```
$ cd /usr/local/corex-{v.r.m}/corex-toolbox-{version}/bin # 以随软件栈同时安装测试工具为例
$ ./p2pBandwidthTest
```

结果矩阵分别为：

- 多卡连接性
- 多卡间打开 P2P 的单向读写带宽
- 多卡间打开 P2P 的双向读写带宽

1.6.2 (虚拟机环境下) P2P 性能提升工具

如果您使用的是虚拟机环境，可以通过 ixgpu-vm-p2p-tool 工具包提升天数智芯加速卡之间 P2P 的传输性能。

1.6.2.1 开始之前

- 请确保您的加速卡已暂停使用。
- 请确保您已将 `ixgpu-vm-p2p_{version}_x86_64.run` 工具包分别下载至您的宿主机和虚拟机环境中。获取 `ixgpu-vm-p2p_{version}_x86_64.run` 工具包的方式有：
 - 登录 [天数智芯官网](#)，进入 **客户支持 > 资源中心** 页面进行下载
 - 联系您的应用工程师获取

1.6.2.2 P2P 性能提升方法

1. 在宿主机中运行 `ixgpu-vm-p2p_{version}_x86_64.run` 工具包：

```
$ bash ixgpu-vm-p2p_{version}_x86_64.run
```

此时，您需要确保 `iluvatar_disable_acs.service` 服务是运行状态。该服务可以确保机器重启后，加速卡间的 P2P 通信性能仍然得到提升。

2. 立即在虚拟机中运行 `ixgpu-vm-p2p_{version}_x86_64.run` 工具包：

```
$ bash ixgpu-vm-p2p_{version}_x86_64.run
```

此时，您需要确保 `iluvatar_vm_p2p.service` 服务是运行状态。该服务可以确保天数智芯加速卡驱动被重新加载后，驱动 P2P 地址表能够自动重新配置，从而确保加速卡间的 P2P 通信性能得到提升。

Important

在虚拟机中运行 **`bash *.run`** 命令之前，请勿加载天数智芯加速卡驱动，即请勿执行 **`modprobe iluvatar`** 操作。

3. 在虚拟机中卸载天数智芯加速卡驱动并重新安装驱动：

```
$ rmmod iluvatar
$ modprobe iluvatar
```

Important

当前操作请务必在运行 **`bash *.run`** 命令之后执行，且只能执行一次，否则需要在宿主机端重新执行 **`bash *.run`** 命令。

4. 通过 P2P 性能测试工具 `p2pBandwidthTest` 验证 P2P 性能是否已提升：

```
# 设置环境变量
$ export PATH=/usr/local/corex-{v.r.m}/bin:$PATH
$ export LD_LIBRARY_PATH=/usr/local/corex-{v.r.m}/lib64:$LD_LIBRARY_PATH

# 测试 P2P 性能
$ cd /usr/local/corex-{v.r.m}/corex-toolbox-{version}/bin # 以随软件栈同时安装测试工具为例
$ ./p2pBandwidthTest
```

1.7 通讯库性能测试工具

ixCCL-tests 工具作为通讯库性能测试工具，用于验证 all gather、all reduce、all to all、broadcast、reduce scatter 等多卡通信 OP 的功能和性能。

ixCCL-tests 是基于 NCCL-tests 在天数智算软件栈环境中重新编译获得，具体的使用方法也可以参考 [NCCL-tests](#)。

1.7.1 开始之前

使用 ixCCL-tests 工具前，您需要确保如下操作已完成：

- 如果需要使用 IB/ROCE 网卡，您需要确保网卡驱动已安装、RDMA 互通测试正常。使用 Docker 容器环境进行测试时，需要在创建 Docker 容器时加上 **--network=host** 参数。
- 您需要确保参与测试的机器都已正确安装天数智算软件栈，并可以运行基本的 GPU 程序。如果需要使用 GDR 作为机间的通信方式，则需要确保已加载 itr_peer_mem_drv 模块，您可以通过 **lsmod | grep itr_peer_mem_drv** 命令进行确认。
- 进行多机通信测试，需要配置 SSH 免密登录。在执行测试前，需要先互相 SSH 登录一次，用于确保 known-hosts 之间已保存了各自的 list。如果环境在 Docker 容器内，则需要确认 SSH 的监听端口，并使用 mpirun 的 MCA 参数 **-mca plm_rsh_args** 来指定，详情请参考 [MCA 参数说明](#)。

注：配置多机的 Docker 容器，如果是非 root 用户想要使用 RDMA，则需要修改 memlock unlimited，即在启动容器时加上 **--ulimit memlock=-1** 参数；如果是主机环境，则需要使用 **ulimit -l unlimited** 命令。

- 请您先设置环境变量 **export NCCL_USE_DIRECT=1** (NCCL_USE_DIRECT 的默认值为 0)。
- 如果需要使用多进程多卡执行测试，需要确保环境中已安装 Open MPI v4.0.7，可以通过 **mpirun -V** 命令查看环境中 MPI 的版本信息。

1.7.2 参数说明

以下是使用 ixCCL-tests 工具进行测试时涉及的参数说明。

1.7.2.1 mpirun 重要参数说明

- **--allow-run-as-root**：允许 mpirun 在 root 用户执行时运行 (mpirun 默认在以 root 用户身份启动时中止)
- **--prefix <dir>**：指定 MPI 的安装路径
- **-np <number of processes>**：指定并行进程的数量
- **-x <variable>**：将环境变量传递给远程主机
- **-H, -host, --host <host1,host2,...,hostN>**：指定要在其上启动进程的主机名，形如：0.113.2.20:4,10.113.2.21:4，“:” 后紧跟的“4” 每个节点启动的进程的上限
- **-mca <key> <value>**：指定 MCA 参数

1.7.2.2 NCCL 环境变量说明

- NCCL_NET: 指定多机通讯的方式可以为 Socket 或者 IB, 指定为 IB 的同时, 加载了 itr_peer_mem_drv 驱动就会通过 GDR 进行通信; 没有加载该驱动则会通过 host IB 的方式进行通信
- NCCL_SOCKET_IFNAME: 指定 Socket 通信使用的网卡
- NCCL_IB_DISABLE: 为 1 则表示禁用 IB 网卡

1.7.2.3 MCA 参数说明

- -mca plm_rsh_args: 指定启动 MPI 进程是传递给 SSH 或者 RSH 的参数, 该参数是个字符串, 形如: -mca plm_rsh_args '-p 55555', 其中的"55555"是 ssh_config 配置的 Port
- -mca btl ^openib: MPI 进程将使用除 OpenIB 之外的所有可用通信模块进行通信
- -mca btl_tcp_if_include bond0 (网卡): 指定在启动 MPI 进程时要使用的通信模块。"btl"表示 Byte Transfer Layer, 是 OpenMPI 中的一个组件, 用于在 MPI 进程之间传输数据; "tcp"表示使用 TCP/IP 协议进行通信; "if_include"表示只使用指定的网络接口进行通信

1.7.2.4 CUDA 环境变量说明

- CUDA_DEVICES_ORDER=PCI_BUS_ID: 要求运行时设备查询按照 PCI_BUS_ID 的顺序索引, 从而使得设备 ID= 物理 ID, 保证 CUDA 应用按期望使用指定设备

1.7.2.5 ixCCL-tests 可执行程序参数说明

- -b,--minbytes <min size in bytes>: 起始值, 默认值为 32M
- -e,--maxbytes <max size in bytes>: 最大值, 默认值为 32M
- -f,--stepfactor <increment factor>: 增量因子, 大小之间的乘法因子, 默认禁用
- -g,--ngpus <GPUs per thread>: 每个线程的 GPU 数量, 默认值为 1

1.7.3 通讯库性能测试方法

场景一: 单机双卡

执行以下命令:

```
$ cd /usr/local/corex-{v.r.m}/corex-toolbox-{version}/bin # 以随软件栈同时安装测试工具为例
$ mpirun --allow-run-as-root --report-bindings -tag-output --prefix /usr/local -np 2
  ↪ --bind-to none --map-by node -mca btl ^openib ./all_reduce_perf -b 8 -e 1G -f 2 -g 1
```

场景二: 单机 8 卡

执行以下命令:

```
$ cd /usr/local/corex-{v.r.m}/corex-toolbox-{version}/bin # 以随软件栈同时安装测试工具为例
$ mpirun --allow-run-as-root --report-bindings -tag-output --prefix /usr/local -np 8
  ↪ --bind-to none --map-by node -mca btl ^openib ./all_reduce_perf -b 8 -e 1G -f 2 -g 1
```

场景三：双机 8 卡

执行以下命令：

```
$ cd /usr/local/corex-{v.r.m}/corex-toolbox-{version}/bin # 以随软件栈同时安装测试工具为例
$ mpirun --allow-run-as-root --report-bindings -tag-output --prefix /usr/local -np 8 -H
↪ 10.113.2.20:4,10.113.2.21:4 --bind-to none --map-by node -x NCCL_NET=IB -x
↪ NCCL_SOCKET_IFNAME=bond0 -x NCCL_IB_DISABLE=0 -x NCCL_ALGO=Ring -x
↪ CUDA_DEVICES_ORDER=PCI_BUS_ID -x LD_LIBRARY_PATH -mca plm_rsh_args '-p 55555' -mca pm1
↪ ob1 -mca btl ^openib -mca btl_tcp_if_include bond0 ./all_reduce_perf -b 8 -e 1G -f 2
↪ -g 1
```

1.8 GDS 基准测试工具

GDS 通过 DMA 使 GPU 与磁盘之间可以直接传输数据，这样可以避免数据通过 CPU，从而提升数据传输带宽并降低 CPU 的负载。天数智芯提供验证 GDS 功能是否可用的工具 ixGDS。

1.8.1 开始之前

使用 ixGDS 工具前，您需要做以下准备工作。

1.8.1.1 NVMe 磁盘与驱动

- NVMe 磁盘

```
# 检查环境中有无 NVMe 磁盘
$ lsblk |grep nvme
nvme0n1          259:0      0 465.8G  0 disk
├─nvme0n1p1      259:1      0   512M  0 part
└─nvme0n1p2      259:2      0 465.3G  0 part /mnt/nvmetest

# 如果磁盘没有挂载，则需要进行挂载操作
$ mount -o data=ordered /dev/nvme0n1p2 /mnt/nvmetest
```

- NVMe 驱动

```
# 检查环境中有无符合要求的 NVMe 驱动
$ lsmod |grep nvme
grep 'nvme_v1_register_nvfs_dma_ops' /proc/kallsyms # Linux 社区中的 NVMe 驱动没有提供
↪ nvme_v1_register_nvfs_dma_ops 函数，因此可以用这个函数来检查驱动是否满足要求
```

如果 NVMe 驱动不符合要求，可以使用安装脚本进行安装。安装脚本使用了 [Linux InfiniBand Drivers \(nvidia.com\)](https://nvidia.com) v5.8 版本。如果您的环境是 Ubuntu 18.04 且内核为 4.15.0，则可以直接使用官方版本的安装包进行安装。

Note

如果当前系统安装在 NVMe 磁盘上，NVMe 驱动则需要安装到 initramfs 中。参考如下操作将 NVMe 驱动安装到 initramfs 中：

(在执行下述命令前，请您先备份原有的 initramfs，避免产生因驱动不可用导致的问题。)

- Ubuntu/Debian:

```
$ echo 'nvme' >> /etc/initramfs-tools/modules
$ update-initramfs -u -k $(uname -r)

# 检查 NVMe 驱动是否加入到 initramfs 中
$ lsinitramfs /boot/initrd.img-$(uname r) | grep nvme
```

- CentOS:

```
$ dracut --add-driver 'nvme' --force

# 检查 NVMe 驱动是否加入到 initramfs 中
$ lsinitrd /boot/initramfs-$(uname r).img | grep nvme
```

1.8.1.2 itrfs 驱动和 libcufile.so 文件

- itrfs 驱动

- itrfs 驱动将随天数智算软件栈 (在安装驱动时) 同时安装。当 NVMe 驱动不符合要求时，安装软件栈时将不会安装 itrfs 驱动。
- 您可以通过 **lsmod | grep itrfs** 命令查询 itrfs 驱动是否存在。
- 在安装 itrfs 时，/etc/modules-load.d/iluvatar.conf 中会配置自动加载 itrfs；iluvatar service 示例会出现 itr_fs: iluvatarcorex-init@iluvatar[,itr_peer_mem_drv],itrfs 提示。

- libcufile.so 文件

软件栈安装目录中存在 COREX_DIR/lib64/libcufile.so.1 文件及其软链接 libcufile.so。

1.8.2 ixGDS 工具使用方法

ixGDS 工具提供 GDSIO 和 GDSCP 两个工具。

1.8.2.1 使用 GDSIO 工具

GDSIO 工具可以根据不同的输入生成 IO 负载。

您可以通过定义配置文件来定义需要产生的 IO 负载，详细参数文档，请参考 [1. fio - Flexible I/O tester rev. 3.36 --- fio 3.36 documentation](#)。

以下是 libcufile-cufile.gdsio 文件的内容参考：

```
[global]
ioengine=libcufile
directory=/mnt
gpu_dev_ids=0
cuda_io=cufile
# 'direct' must be 1 when using cuda_io=cufile
direct=1
# Performance is negatively affected if 'bs' is not a multiple of 4k.
# Refer to GDS cuFile documentation.
bs=4K
size=4g
io_size=4g
numjobs=1

# cudaMalloc fails if too many processes attach to the GPU, use threads.
thread

[read]
rw=read

[write]
rw=write
stonewall
```

此外，您也可以通过命令行的方式定义负载。上述示例使用命令行定义负载，即为：

```
$ gdsio --ioengine=libcufile --directory=/mnt --direct=1 --bs=4K --size=4g --io_size=4g
↪ --numjobs=1 --thread --gpu_dev_ids=0 --cuda_io=cufile --name=read --rw=read
↪ --name=write --rw=write --stonewall
```

您可以通过 **./gdsio --cmdhelp** 命令查看完整的 option 信息，在 cuFile 主要为：

- rw，执行读/写
- bs，每次执行的 IO 大小，需要 4K 对齐 (与 PAGESIZE 对齐)
- size，一个 job 总大小
- io_size job，实际执行的总大小 (io_size 应当不大于 size)

1.8.2.2 使用 GDSCP 工具

GDSCP 工具使用 cuFile API 实现了拷贝文件的功能，可以检查当前环境是否支持 GDS，输入文件也需要 4K 对齐。

下面是一个使用 GDSCP 工具的示例：

```
$ ./gdscp --help
./gdscp <readfilepath> <writefilepath> <gpuid>

$ ./gdscp /mnt/nvmetest/fio_test/cp_source /mnt/nvmetest/fio_test/cp_dest_0223 0
```

```
gpu md5sum:50991d699f48bc6aba25fae3e05d3329
input md5sum:50991d699f48bc6aba25fae3e05d3329
output md5sum:50991d699f48bc6aba25fae3e05d3329
MD5 matched.
```

1.9 GPU 性能调试工具

天数智芯提供 GPU 性能调试工具 ix-gpu-tuner，该工具是一个综合性诊断工具，用于检查影响 D2H (Device to Host, 设备到主机), H2D (Host to Device, 主机到设备), P2P (Peer to Peer, 点到点) 数据传输的系统设置和 PCIe 配置以及 GPU 状态。

1.9.1 使用方法

ix-gpu-tuner 工具的命令行格式为：

```
$ ./ix-gpu-tuner [OPTIONS]
```

以下是 ix-gpu-tuner 提供的命令行选项及对应说明。

选项	描述	允许值
-l, --list	显示 GPU 的基础信息	NULL
--topo TEXT	显示 PCIe 拓扑结构 v: 不完整的 switch 结构且不含设备 busID vv: 不完整的 switch 结构但显示 busID vvv: 显示 busID 并显示完整的 switch 结构	v, vv, vvv
--trace	检测引起通讯的阻塞或降速的因素 1. iommu 状态 2. pcie switch 状态 3. switch acs 状态 4. pcie 扩展功能状态 5. GPU Completion Timeout Disable 状态	NULL
--disable-acs	关闭 GPU 链路上所有 switch 的 acs 功能	NULL
--enable-acs	开启 GPU 链路上所有 switch 的 acs 功能	NULL

选项	描述	允许值
--enable-extend	使能 PCIe 扩展能力	NULL
--set-mps TEXT	设置 GPU MaxReadReq Size 0: 128B 1: 256B 2: 512B 3: 1024B 4: 2048B 5: 4096B	0, 1, 2, 3, 4, 5
--set-timeoutDis TEXT	设置 GPU Completion Timeout Disable 状态 0: Disable 1: Enable	0, 1

1.9.2 使用示例

1.9.2.1 示例 1：使用 -l, --list 查看 GPU 的状态

```
$ ./ix-gpu-tuner -l
```

输出结果示例如下图所示：

```
root@ 5:/home/ # ./ix-gpu-tuner -l
-----
NUM | DEVICE          | PRODUCT | NUMA_Affinity | CPU_Affinity
-----
0   | 0000:1d:00.0    | MR-V100 | 0             | 0-19,40-59
1   | 0000:1e:00.0    | MR-V100 | 0             | 0-19,40-59
2   | 0000:21:00.0    | MR-V100 | 0             | 0-19,40-59
3   | 0000:25:00.0    | MR-V100 | 0             | 0-19,40-59
-----
root@ 5:/home/ #
```

图 1: 查看 GPU 状态输出结果示例

1.9.2.2 示例 2：使用 --topo 参数查看 PCIe 拓扑图

1. 显示不完整的 switch 结构且不含设备 busID:

```
$ ./ix-gpu-tuner --topo -v
```

输出结果示例如下图所示：



图 2: 查看 PCIe 拓扑图输出结果示例 1

2. 显示不完整的 switch 结构但显示 busID:

```
$ ./ix-gpu-tuner --topo -vv
```

输出结果示例如下图所示:

```
[NUMA 0] -- [Root Port] 03:01.0 -- [SWITCH] 04:00.0 +- [SWITCH] 06:00.0 -- [SWITCH] 08:00.0 +- [SWITCH] 0a:00.0 -- [GPU] 0c:00.0
|                                     |
|                                     \- [SWITCH] 0d:00.0 -- [GPU] 0f:00.0
|
| \- [SWITCH] 15:00.0 -- [SWITCH] 17:00.0 +- [SWITCH] 19:00.0 -- [GPU] 1b:00.0
|                                     |
|                                     \- [SWITCH] 1c:00.0 -- [GPU] 1e:00.0

[NUMA 0] -- [Root Port] 38:01.0 -- [SWITCH] 39:00.0 +- [SWITCH] 3b:00.0 -- [SWITCH] 3d:00.0 +- [SWITCH] 3f:00.0 -- [GPU] 41:00.0
|                                     |
|                                     \- [SWITCH] 42:00.0 -- [GPU] 44:00.0
|
| \- [SWITCH] 4a:00.0 -- [SWITCH] 4c:00.0 +- [SWITCH] 4e:00.0 -- [GPU] 50:00.0
|                                     |
|                                     \- [SWITCH] 51:00.0 -- [GPU] 53:00.0

[NUMA 1] -- [Root Port] 81:01.0 -- [SWITCH] 82:00.0 +- [SWITCH] 84:00.0 -- [SWITCH] 86:00.0 +- [SWITCH] 88:00.0 -- [GPU] 8a:00.0
|                                     |
|                                     \- [SWITCH] 8b:00.0 -- [GPU] 8d:00.0
|
| \- [SWITCH] 92:00.0 -- [SWITCH] 94:00.0 +- [SWITCH] 96:00.0 -- [GPU] 98:00.0
|                                     |
|                                     \- [SWITCH] 99:00.0 -- [GPU] 9b:00.0

[NUMA 1] -- [Root Port] b7:01.0 -- [SWITCH] b8:00.0 +- [SWITCH] ba:00.0 -- [SWITCH] bc:00.0 +- [SWITCH] be:00.0 -- [GPU] c0:00.0
|                                     |
|                                     \- [SWITCH] c1:00.0 -- [GPU] c3:00.0
|
| \- [SWITCH] c8:00.0 -- [SWITCH] ca:00.0 +- [SWITCH] cc:00.0 -- [GPU] ce:00.0
|                                     |
|                                     \- [SWITCH] cf:00.0 -- [GPU] d1:00.0

Legend:
NUMA      : Non-Uniform Memory Access node.
Root Port : Located in the Root Complex and serves as the starting point of the bus topology.
Switch    : Contains one upstream port and one downstream port. The displayed busID corresponds to the upstream port.
Up Port   : PCIe switch upstream port.
Down Port : PCIe switch downstream port.
GPU       : IX GPU.
```

图 3: 查看 PCIe 拓扑图输出结果示例 2

3. 显示 busID 并显示完整的 switch 结构:

```
$ ./ix-gpu-tuner --topo -vvv
```

输出结果示例如下图所示:

```
[NUMA 0] -- [Root Port] 03:01.0 -- [Up Port] 04:00.0 +- [Down Port] 05:00.0 -- [Up Port] 06:00.0 -- [Down Port] 07:00.0 -- [Up Port] 08:00.0 +- [Down Port] 09:00.0 -- [Up Port] 0a:00.0 -- [Down Port] 0b:00.0 -- [GPU] 0c:00.0
|                                     |
|                                     \- [Down Port] 09:04.0 -- [Up Port] 0d:00.0 -- [Down Port] 0e:00.0 -- [GPU] 0f:00.0
|
| \- [Down Port] 05:08.0 -- [Up Port] 15:00.0 -- [Down Port] 16:10.0 -- [Up Port] 17:00.0 +- [Down Port] 18:00.0 -- [Up Port] 19:00.0 -- [Down Port] 1a:00.0 -- [GPU] 1b:00.0
|                                     |
|                                     \- [Down Port] 18:04.0 -- [Up Port] 1c:00.0 -- [Down Port] 1d:00.0 -- [GPU] 1e:00.0

[NUMA 0] -- [Root Port] 38:01.0 -- [Up Port] 39:00.0 +- [Down Port] 3a:00.0 -- [Up Port] 3b:00.0 -- [Down Port] 3c:00.0 -- [Up Port] 3d:00.0 +- [Down Port] 3e:00.0 -- [Up Port] 3f:00.0 -- [Down Port] 40:00.0 -- [GPU] 41:00.0
|                                     |
|                                     \- [Down Port] 3e:04.0 -- [Up Port] 42:00.0 -- [Down Port] 43:00.0 -- [GPU] 44:00.0
|
| \- [Down Port] 3a:08.0 -- [Up Port] 4a:00.0 -- [Down Port] 4b:10.0 -- [Up Port] 4c:00.0 +- [Down Port] 4d:00.0 -- [Up Port] 4e:00.0 -- [Down Port] 4f:00.0 -- [GPU] 50:00.0
|                                     |
|                                     \- [Down Port] 4d:04.0 -- [Up Port] 51:00.0 -- [Down Port] 52:00.0 -- [GPU] 53:00.0

[NUMA 1] -- [Root Port] 81:01.0 -- [Up Port] 82:00.0 +- [Down Port] 83:00.0 -- [Up Port] 84:00.0 -- [Down Port] 85:00.0 -- [Up Port] 86:00.0 +- [Down Port] 87:00.0 -- [Up Port] 88:00.0 -- [Down Port] 89:00.0 -- [GPU] 8a:00.0
|                                     |
|                                     \- [Down Port] 87:04.0 -- [Up Port] 8b:00.0 -- [Down Port] 8c:00.0 -- [GPU] 8d:00.0
|
| \- [Down Port] 83:08.0 -- [Up Port] 92:00.0 -- [Down Port] 93:10.0 -- [Up Port] 94:00.0 +- [Down Port] 95:00.0 -- [Up Port] 96:00.0 -- [Down Port] 97:00.0 -- [GPU] 98:00.0
|                                     |
|                                     \- [Down Port] 95:04.0 -- [Up Port] 99:00.0 -- [Down Port] 9a:00.0 -- [GPU] 9b:00.0

[NUMA 1] -- [Root Port] b7:01.0 -- [Up Port] b8:00.0 +- [Down Port] b9:00.0 -- [Up Port] ba:00.0 -- [Down Port] bb:00.0 -- [Up Port] bc:00.0 +- [Down Port] bd:00.0 -- [Up Port] be:00.0 -- [Down Port] bf:00.0 -- [GPU] c0:00.0
|                                     |
|                                     \- [Down Port] bd:04.0 -- [Up Port] c1:00.0 -- [Down Port] c2:00.0 -- [GPU] c3:00.0
|
| \- [Down Port] b9:08.0 -- [Up Port] c8:00.0 -- [Down Port] c9:10.0 -- [Up Port] ca:00.0 +- [Down Port] cb:00.0 -- [Up Port] cc:00.0 -- [Down Port] cd:00.0 -- [GPU] ce:00.0
|                                     |
|                                     \- [Down Port] cb:04.0 -- [Up Port] cf:00.0 -- [Down Port] d0:00.0 -- [GPU] d1:00.0

Legend:
NUMA      : Non-Uniform Memory Access node.
Root Port : Located in the Root Complex and serves as the starting point of the bus topology.
Switch    : Contains one upstream port and one downstream port. The displayed busID corresponds to the upstream port.
Up Port   : PCIe switch upstream port.
Down Port : PCIe switch downstream port.
GPU       : IX GPU.
```

图 4: 查看 PCIe 拓扑图输出结果示例 3

1.9.2.3 示例 3: 使用 --trace 检测引起通讯的阻塞或降速的因素

```
$ ./ix-gpu-tuner --trace
```

输出结果示例如下图所示:

```
root@1:/home/ # ./ix-gpu-tuner --trace
[IOMMU State]: OFF

[Broadcom Switch PEX89]: Base Mode

[ACS State]: OFF

[PCIe Extend Capabilities]: Enable
```

图 5: 检测引起通讯阻塞或降速的因素输出结果示例

说明:

1. 当存在一个 switch mode 为 synthetic mode 时, trace 结果即会显示为 Synthetic Mode。当只有所有的 switch mode 都为 base mode 时, 才会显示 Base Mode。
2. 当存在一个 acsctl 状态为 enable, trace 结果即会显示为 ON。只有当所有 acsctl 的状态都是 disable 时, 才会显示为 OFF。
3. 当存在一个扩展能力为 disable, trace 结果即会显示为 Disable。只有当所有的扩展能力寄存器都是 enable 时, 才会显示为 Enable。

Tip

博通 switch 模式可能存在以下问题: 如果主板 switch 是 博通 89 系列, 其模式将被检测。**synthetic mode** 会导致天垓 150 加速卡和智铠 100 DUO 加速卡的 P2P 异常。因此正常情况下, 天垓 150 加速卡和智铠 100 DUO 加速卡需要配合 base mode 使用。

您可以通过升级 ssmode 固件版本避免上述问题。博通在 2025 年 2 月发布的 ssmode arctic_12_18247_dcsgAtlasExt (4.18) 固件版本已修复上述问题。如果您有任何问题, 也可以联系您的天数智芯应用工程师获取帮助。

只有 博通 PEX89 switch 才会显示 switch 的模式, PEX88 系列不会显示。

当天数智芯加速卡出现掉卡现象时, 在枚举加速卡设备时会显示出来, 即显示 GPU OFFLINE。如下图所示:

```
root@2:/home/ # ./ix-gpu-tuner --trace
[0000:3b:00.0]: GPU OFFLINE

[IOMMU State]: ON

[0000:87:00.0 ACS]: ON
[0000:87:04.0 ACS]: ON
[0000:89:00.0 ACS]: ON
[0000:8c:00.0 ACS]: ON
[ACS State]: ON

[PCIe Extend Capabilities]: Enable

root@2:/home/ # |
```

图 6: 出现加速卡掉卡现象输出结果示例

1.9.2.4 示例 4：使用 --disable-accs 关闭 switch 的 accs

```
$ ./ix-gpu-tuner --disable-accs
```

输出结果示例如下图所示：

```
root@_____1:/home/|_____# ./ix-gpu-tuner --disable-accs
[ACS State]: OFF

root@_____1:/home/|_____# |
```

图 7: 关闭 switch 的 accs 输出结果示例

1.9.2.5 示例 5：使用 --enable-accs 开启 switch 的 accs

```
$ ./ix-gpu-tuner --enable-accs
```

输出结果示例如下图所示：

```
root@_____:/home/|_____# ./ix-gpu-tuner --enable-accs
[ACS State]: ON

root@_____:/home/|_____#
```

图 8: 开启 switch 的 accs 输出结果示例

1.9.2.6 示例 6：使用 --enable-extend 使能 PCIe 扩展能力

```
$ ./ix-gpu-tuner --enable-extend
```

输出结果示例如下图所示：

```
root@_____1:/home/|_____# ./ix-gpu-tuner --enable-extend
[PCIe Extend Capabilities]: Enable

root@_____1:/home/|_____#
```

图 9: 使能 PCIe 扩展能力输出结果示例

1.9.2.7 示例 7：使用 --set-mps 设置 MaxPayload Size

```
$ ./ix-gpu-tuner --set-mps 1
```

输出结果示例如下图所示：

```
root@ 5:/home/ # ./ix-gpu-tuner --set-mps 1
[MaxPayload Size]: 256B
root@ 5:/home/ # |
```

图 10: 设置 MaxPayload Size 输出结果示例

1.9.2.8 示例 8: 使用 --set-mrrs 设置 MaxReadReq Size

```
$ ./ix-gpu-tuner --set-mrrs 1
```

输出结果示例如下图所示：

```
root@ 5:/home/ # ./ix-gpu-tuner --set-mrrs 1
[MaxReadReq Size]: 4096B
root@ 5:/home/ # |
```

图 11: 设置 MaxReadReq Size 输出结果示例

1.9.2.9 示例 9: 使用 --set-timeoutDis 设置 Completion Timeout Disable

```
$ ./ix-gpu-tuner --set-timeoutDis 1
```

输出结果示例如下图所示：

```
root@ 5:/home/ # ./ix-gpu-tuner --set-timeoutDis 1
[Completion Timeout Disable]: Enable
root@ 5:/home/ # |
```

图 12: 设置 Completion Timeout Disable 输出结果示例

1.10 (第三方类 FPGA 设备) DMA 测试工具

天数智芯提供 DMA 测试工具 ixgpu_to_third_party_dmatetest，用于读写用户设备的内存。

以下是用户设备和天数智芯加速卡的内存读写场景：

- 用户设备为发起方 (Master)，天数智芯加速卡为接收方 (Slaver)

比如，用户设备为 FPGA：FPGA 直接向天数智芯加速卡内存做 DMA 读写操作。

该场景，无需使用 DMA 测试工具，但需要用户的驱动程序调用天数 KMD 驱动的 **RDMA 公共 API 接口**。

- 天数智芯加速卡为发起方 (Master)，用户设备为接收方 (Slaver)

比如，用户设备为 FPGA：天数智芯加速卡直接向 FPGA 内存做 DMA 读写操作。

该场景，需要用到 DMA 测试工具。

下面对两种场景对应的操作分别进行详细说明。

1.10.1 场景 1：用户设备为 Master，天数智芯加速卡为 Slaver

在该场景下，无需使用 DMA 测试工具。

1. 请确保您的环境中存在类 FPGA 设备驱动。驱动示例参考程序如下：

```
int main(void)
{
    int fd_nv, fd_rdma;
    void *mem_h, *mem_d;
    struct ioctl_allocation_args alloc_args_h, alloc_args_d;
    struct ioctl_map_memory_to_gpu_args map_args_h, map_args_d;
    struct rdma_ioctl_memcpy_args dma_args;
    int ret;

    if ((fd_nv = open(NV_DEV, O_RDWR)) < 0) {
        printf("open nv device failed\n");
        ret = fd_nv;
        goto err_open_nv;
    }

    if ((fd_rdma = open(RDMA_DEV, O_RDWR)) < 0) {
        printf("open rdma device failed\n");
        ret = fd_rdma;
        goto err_open_rdma;
    }

    printf("Device opened: fd (%d, %d)\n", fd_nv, fd_rdma);

    /* allocate nv device hbm */
    alloc_args_d.len = SZ_64M;
    alloc_args_d.mem_type = NV_MEMORY_TYPE_UMA_4K;
    alloc_args_d.heap_type = NV_HEAP_TYPE_HBM;
    alloc_args_d.flags = NV_FLAG_UNCACHED;

    ret = ioctl(fd_nv, NV_IOC_ALLOC, &alloc_args_d);
    if (ret) {
        printf("allocate device hbm failed\n");
        goto err_alloc_dev;
    }

    mem_d = mmap(NULL, alloc_args_d.len, PROT_WRITE | PROT_READ, MAP_SHARED,
        alloc_args_d.fd, 0);
```

```

memset(mem_d, 0, alloc_args_d.len);

/* map to gpu in order to find ion buffer with address later */
map_args_d.addr = (uint64_t)mem_d;
map_args_d.len = alloc_args_d.len;
map_args_d.fd = alloc_args_d.fd;

ret = ioctl(fd_NV, NV_IOC_MAP_MEMORY_TO_GPU, &map_args_d);
if (ret) {
    printf("map device hbm to gpu failed\n");
    goto err_map_dev;
}

/* allocate system memory */
alloc_args_h.len = SZ_32M;
alloc_args_h.mem_type = NV_MEMORY_TYPE_NULL;
alloc_args_h.heap_type = NV_HEAP_TYPE_SYSTEM;
alloc_args_h.flags = NV_FLAG_CACHED;

ret = ioctl(fd_NV, NV_IOC_ALLOC, &alloc_args_h);
if (ret) {
    printf("allocate system memory failed\n");
    goto err_alloc_host;
}

mem_h = mmap(NULL, alloc_args_h.len, PROT_WRITE | PROT_READ, MAP_SHARED,
             alloc_args_h.fd, 0);
memset(mem_h, 0, alloc_args_h.len);

/* map to gpu in order to find ion buffer with address later */
map_args_h.addr = (uint64_t)mem_h;
map_args_h.len = alloc_args_h.len;
map_args_h.fd = alloc_args_h.fd;

ret = ioctl(fd_NV, NV_IOC_MAP_MEMORY_TO_GPU, &map_args_h);
if (ret) {
    printf("map system memory to gpu failed\n");
    goto err_map_host;
}

/*
 * start H2D dma copy
 */
dma_args.src_addr = (uint64_t)mem_h;
dma_args.dst_addr = (uint64_t)mem_d;
dma_args.len = alloc_args_d.len;
dma_args.dir = RDMA_MEMCPY_H2D;

printf("dump hbm sg_table, please refer to dmesg...\n");
ret = ioctl(fd_rdma, RDMA_SKELETON_IOC_MEMCPY, &dma_args);

```

```

if (ret) {
    printf("H2D dma copy failed\n");
    goto err_copy;
}

/* clean works */
ioctl(fd_nv, NV_IOC_UNMAP_MEMORY_FROM_GPU, &map_args_h);
ioctl(fd_nv, NV_IOC_UNMAP_MEMORY_FROM_GPU, &map_args_d);
munmap(mem_h, alloc_args_h.len);
close(alloc_args_h.fd);
munmap(mem_d, alloc_args_d.len);
close(alloc_args_d.fd);
close(fd_nv);

return 0;

err_copy:
    ioctl(fd_nv, NV_IOC_UNMAP_MEMORY_FROM_GPU, &map_args_h);
err_map_host:
    munmap(mem_h, alloc_args_h.len);
    close(alloc_args_h.fd);
err_alloc_host:
    ioctl(fd_nv, NV_IOC_UNMAP_MEMORY_FROM_GPU, &map_args_d);
err_map_dev:
    munmap(mem_d, alloc_args_d.len);
    close(alloc_args_d.fd);
err_alloc_dev:
    close(fd_rdma);
err_open_rdma:
    close(fd_nv);
err_open_nv:
    return ret;
}

```

2. 新建一个 UMD 程序文件并插入相关接口代码，参考示例如下：

```

#include <fcntl.h>
#include <stdio.h>
#include <sys/ioctl.h>
#include <unistd.h>

#include <regex>
#include <string>

#include <cuda_runtime.h>

using u64 = unsigned long long;

#define checkCudaErrors(a)
    ↪ \
    do
    ↪ \

```

```

    {
↪ \
        auto temp_result = a;
↪ \
        if (0 != (int)(temp_result))
↪ \
        {
↪ \
            fprintf(stderr,
↪ \
                "%s:%d %s error number is %d. Runtime error string: %s\n",
↪ \
                __FILE__,
↪ \
                __LINE__,
↪ \
                #a,
↪ \
                temp_result,
↪ \
                cudaGetErrorString(cudaGetLastError()));
↪ \
            fflush(stderr);
↪ \
            exit(EXIT_FAILURE);
↪ \
        }
↪ \
    } while (0);

#define ITR_PEERM_TEST 1
#define ITR_PEERM_TEST_IOCTL _IOWR(0x82, ITR_PEERM_TEST, struct ItrPeemIoctlTestArgs)

const std::string dev_name("/dev/itr_peerm_dev0"); // 用户的 FPGA 设备

struct ItrPeemIoctlTestArgs
{
    u64 va;
    u64 size;
};

int RunRdmaTest(void* va, u64 size)
{
    int rc;

    int fd = open(dev_name.c_str(), O_RDWR);
    if (fd < 0)
    {
        printf("    failed to open device: %s\n", dev_name.c_str());
        return 1;
    }
}

```

```

    }

    ItrPeemIoctlTestArgs ioc_input = {0};
    ioc_input.va                  = (u64)va;
    ioc_input.size                = size;
    rc                           = ioctl(fd, ITR_PEERM_TEST_IOCTL, &ioc_input);
    if (rc)
    {
        printf("    rdma test failed!, rc = %d\n", rc);
        return rc;
    }
    return 0;
}

int main(int n, char* args[])
{
    int cudaDeviceId = 0;
    u64 testSize     = (u64)sysconf(_SC_PAGE_SIZE) << 4;

    printf("Cuda Device: %d\n", cudaDeviceId);
    printf("Test Size: 0x%llx\n", testSize);

    checkCudaErrors(cudaSetDevice(cudaDeviceId));

    void* devMem = nullptr;
    checkCudaErrors(cudaMalloc(&devMem, testSize));
    checkCudaErrors(cudaMemset(devMem, 0, testSize));

    printf("Begin to run rdma test\n");
    const int ret = RunRdmaTest(devMem, testSize);
    if (ret == 0)
        printf("Test result: [pass]\n");
    else
        printf("Test result: [fail]\n");

    return ret;
}

```

其中，/dev/itr_peerm_dev0 表示用户设备。

以下是对插入的接口代码的流程说明：

- 用户程序使用 cudaMalloc 完成内存分配。
- 用户程序使用 cudaMemset(devPtr, 0, size) 完成清零。
- 用户程序使用 cudaMallocHost 完成主机内存分配，可用 memset 直接清零。
- 用户驱动需要实现自己的 ioctl 接口 (RDMA_SKELETON_IOC_MEMCPY) 用于完成和用户程序的交互。
 - 将 src, dst, size, direction 带给用户驱动接口。
 - 用户驱动使用 rdma_itr_p2p_get_pages 获取 GPU VA 的 memory ctx。
 - 用户驱动使用 rdma_itr_p2p_dma_map_pages 获取 sg_table。
 - 用户驱动利用 sg_table 进行 DMA，并确保 DMA 完成。

- e) 用户驱动使用 `rdma_itr_p2p_dma_unmap_pages` 和 `rdma_itr_p2p_put_pages` 释放资源。
- e. 用户程序调用 `cudaFree` 释放显存。
- f. 用户程序调用 `cudaFreeHost` 释放主机内存。

您可以参考 [Mellanox 直接向远程 GPU 内存实现 RDMA 读写操作代码示例](#) 了解更多操作说明。

1.10.1.1 RDMA 公共 API 接口

天数智芯提供以下 RDMA 公共 API 供用户调用。

1.10.1.1.1 `rdma_itr_p2p_get_pages`

功能概述

创建一个 GPU 内存上下文。

函数原型

```
int rdma_itr_p2p_get_pages(unsigned long addr, size_t size, void **ctx);
```

参数说明

- `addr`: GPU 虚拟地址
- `size`: GPU 内存大小
- `ctx`: GPU 内存上下文，请务必释放

返回值

int:

- 0: 成功
- `ERRNO`: 失败

1.10.1.1.2 `rdma_itr_p2p_put_pages`

功能概述

释放 GPU 内存上下文。

函数原型

```
void rdma_itr_p2p_put_pages(void *ctx);
```

参数说明

- `ctx`: GPU 内存上下文

返回值

void

1.10.1.1.3 rdma_itr_p2p_dma_map_pages

功能概述

创建一个 DMA 映射 sg_table。

函数原型

```
int rdma_itr_p2p_dma_map_pages(struct device *dma_device, void *sg_head, void *ctx);
```

参数说明

- dma_device: 需要访问 GPU 内存的设备
- sg_head: sg_table 保存 P2P 访问的 DMA 地址列表
- ctx: rdma_itr_p2p_get_pages 创建的上下文

返回值

int:

- 0: 成功
- ERRNO: 失败

1.10.1.1.4 rdma_itr_p2p_dma_unmap_pages

功能概述

取消 DMA 映射的 sg_table。

函数原型

```
int rdma_itr_p2p_dma_unmap_pages(struct device *dma_device, void *sg_head, void *ctx);
```

参数说明

- dma_device: 需要访问 GPU 内存的设备
- sg_head: sg_table 保存 P2P 访问的 dma 地址列表
- ctx: rdma_itr_p2p_get_pages 创建的上下文

返回值

int:

- 0: 成功
- ERRNO: 失败

1.10.2 场景 2: 天数智芯加速卡为 Master, 用户设备 (类 FPGA 设备) 为 Slaver

在该场景下, 需要使用 DMA 测试工具。

Note

- 可以向您的应用工程师获取 FPGA 读写测试 repo。
- 向您的应用工程师获取 DMA 测试工具 `ixgpu_to_third_party_dmatest`。
- 如果您的环境上存在多个天数智芯加速卡，默认是以天数智芯加速卡 `device0` 作为 Master 访问用户设备的内容。

1. 参考《软件栈安装指南》安装天数智算软件栈。

2. 将获取到的 DMA 测试工具 `ixgpu_to_third_party_dmatest` 拷贝至任意目录下。

3. 通过 `lspci -vvs <bus-id> | grep Reg` 命令查询用户设备 Slaver 的 memory bar。

比如，下面这块设备的 bus-id 是 68:00.0，根据 `lspci` 命令查询的结果是 memory bar (Region 1) 是 16G，对应设备文件夹下的 `resource1` 文件，读写 `resource1` 即读写内存：

```
$ lspci -vvs 68:00.0 | grep Reg
Region 0: Memory at d7000000 (32-bit, non-prefetchable) [size=16M]
Region 1: Memory at 1780000000 (64-bit, prefetchable) [size=16G]
Region 3: Memory at 17c0000000 (64-bit, prefetchable) [size=32M]
```

如果 Region 处于 disable 状态，需要执行以下命令打开设备的 IO 和 memory 读写权限：

```
setpci -s 68:00.0 COMMAND=0x06
```

4. 比如，要测试 4G 大小的 DMA，执行以下命令即可：

```
$ ./ixgpu_to_third_party_dmatest /sys/bus/pci/devices/${0000\:68\:00.0}/${resource1}
↪ ${0x100000000}
# 其中，0000\:68\:00.0 为设备的 bus id, resource1 即需要读写的 region, 0x100000000 为一次读
↪ 写的 data size (16 进制)
# 请您根据实际环境配置以上参数
```

参考示例如下：

```
$ ./gpu_to_third_party.bin /sys/bus/pci/devices/0000\:68\:00.0/resource1 0x100000000
Open path is /sys/bus/pci/devices/0000:68:00.0/resource1, mmap offset is 0x0, mmap
↪ size is 0x100000000
Open succeed
mmap succeed
The cpu ptr returned by mmap is 0x7fcf2fd92000
Test DMA, function: TestGPUThirdPartyDeviceDMA_V2
GPU -> Third party device speed is 9.557e+00 GB/s
Third party device -> GPU speed is 2.899e+00 GB/s
Pass!
Test kernel add on third party device memory! function: TestKernelAdd
u8:Pass!
u16:Pass!
u32:Pass!
u64:Pass!
s8:Pass!
s16:Pass!
s32:Pass!
s64:Pass!
float:Pass!
Test result: PASS
```

商标声明

- 天数智芯、天数智芯 logo、Iluvatar CoreX 等商标、标识、组合商标为上海天数智芯半导体有限公司之注册商标或商标，受法律保护。
- CentOS 标识为 Red Hat 公司的商标。
- Docker 为 Docker 公司在美国和其他国家的商标或注册商标。
- Linux 为 Linus Torvalds 在美国和其它国家的注册商标。
- NVIDIA、CUDA、GeForce 和 GeForce GTX 为 NVIDIA 公司在美国和/或其它国家的商标和/或注册商标。
- PyTorch 为 Facebook 公司的商标。
- TensorFlow 为 Google 公司的商标。
- Ubuntu 为 Canonical 公司的注册商标。

除上述明确列出的商标外，本内容中可能存在的任何其他未明示的产品名称、服务名称、企业名称、标识、口号、域名或其他商业标记 (无论是否标注 ™ 或 ® 符号)，均假定归属于其各自合法的商标权人所有。